20/8/24

# Exp - 7

## Flow Control At Data Link Layer Using Sliding Window Protocol

**Aim:**

To write a program to implement flow control at the data link layer using the sliding window protocol and to simulate the flow the frames from one node to another.

**Algorithm:**

1. Start the program
2. Read the total number of frames to be sent (N), the sender window size (w), and optionally a frame number to simulate as lost during transmission.
3. Initialize sent = 0 (sequence number of the already acknowledged frame) and next seq = 0 (sequence number of the next frame to be sent)
4. Repeat while sent < N:
   a) determine the current sender window as the range [sent, sent + w - 1].
   b) Send all frames with sequence numbers from next seq up to (sent + w - 1), incrementing next seq after each frame at any time.
   c) If the lost frame lies within the frames just sent.


SLIDING WINDOW PROTOCOL – FLOW CONTROL SIMULATION (Go-Back-N)
Enter total number of frames to send : 8
Enter sender window size : 4
Enter frame number to simulate as LOST
(0 to 7, enter -1 for no loss) : 5
Sender Window : [Frame 0 - Frame 3]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node A --> Sending Frame 3
Node B --> Received Frame 0, sending ACK 0
Node B --> Received Frame 1, sending ACK 1
Node B --> Received Frame 2, sending ACK 2
Node B --> Received Frame 3, sending ACK 3
Node A --> ACK 3 received. Sliding window forward.
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
(Frame 5 LOST in transit!)
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Frame 5 missing. Discarding out-of-order frames after it.
Node A --> Timeout! Resending from Frame 5.
Sender Window : [Frame 5 - Frame 7]
Node A --> Sending Frame 5
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Received Frame 5, sending ACK 5
Node B --> Received Frame 6, sending ACK 6
Node B --> Received Frame 7, sending ACK 7
Node A --> ACK 7 received. Sliding window forward.
All 8 frames successfully transmitted from Node A to Node B.

(ii) On receiving the ACK for the last frame sent, the sender slides its window forward.
base = next_seq

(5) Repeat step 4 until all 10 frames have been transmitted and acknowledged

(6) Stop the program

PROGRAM

```
def simulate(total_frames, window_size, lost_fra
    base = 0
    next_seq = 0
    while base < total_frames:
        window_end = min(base + window_size - 1,
        print("\n----------")
        print(f"Sender Window: [frame {base} - Frame
                                {window_end}]")
    # send all frames within the current window
    that haven't been sent.
    while next_seq < base + window_size and
        next_seq < total_frames:
        if next_seq == lost_frame:
            print(f"Frame {next_seq} lost in transit!")
    next_seq += 1
    if lost_frame is not None and base <=
    lost_frame < next_seq:
    # a frame in this batch and base <= lost
        frame = next_seq
```

## Output

=== Sliding Window Protocol - Flow Control Simulation (Go-Back-N) ===

Enter total number of frames to send: 8

Enter sender's window size : 4

Enter frame number to simulate as lost (0 to 7, enter -1 for no loss) : 5

---

Sender Window : [Frame 0 - Frame 3]

Node A → Sending frame 0

Node A → Sending frame 1

Node A → Sending frame 2

Node A → Sending frame 3

Node B → received frame 0, sending ACK 0

Node B → received frame 1, sending ACK

Node B → Rcvd frame 3, sending ACK

Node A → ACK 3 rcvd sliding window forward

---

Sender window : [Frame 4 - frame 7]

Node A → sending frame 4

Node A → Sending Frame 5

(Frame 5 lost in transit!)

Node A → Sending frame 6

Node B → Frame 5 missing. discarding

out of order frames after it.

Node A → Timeout! resending window

starting from frame 5.

Result:

The program to implement error detection and correction using the Hamming code concept was successfully written in Python, executed and executed verified using a sample input data stream.